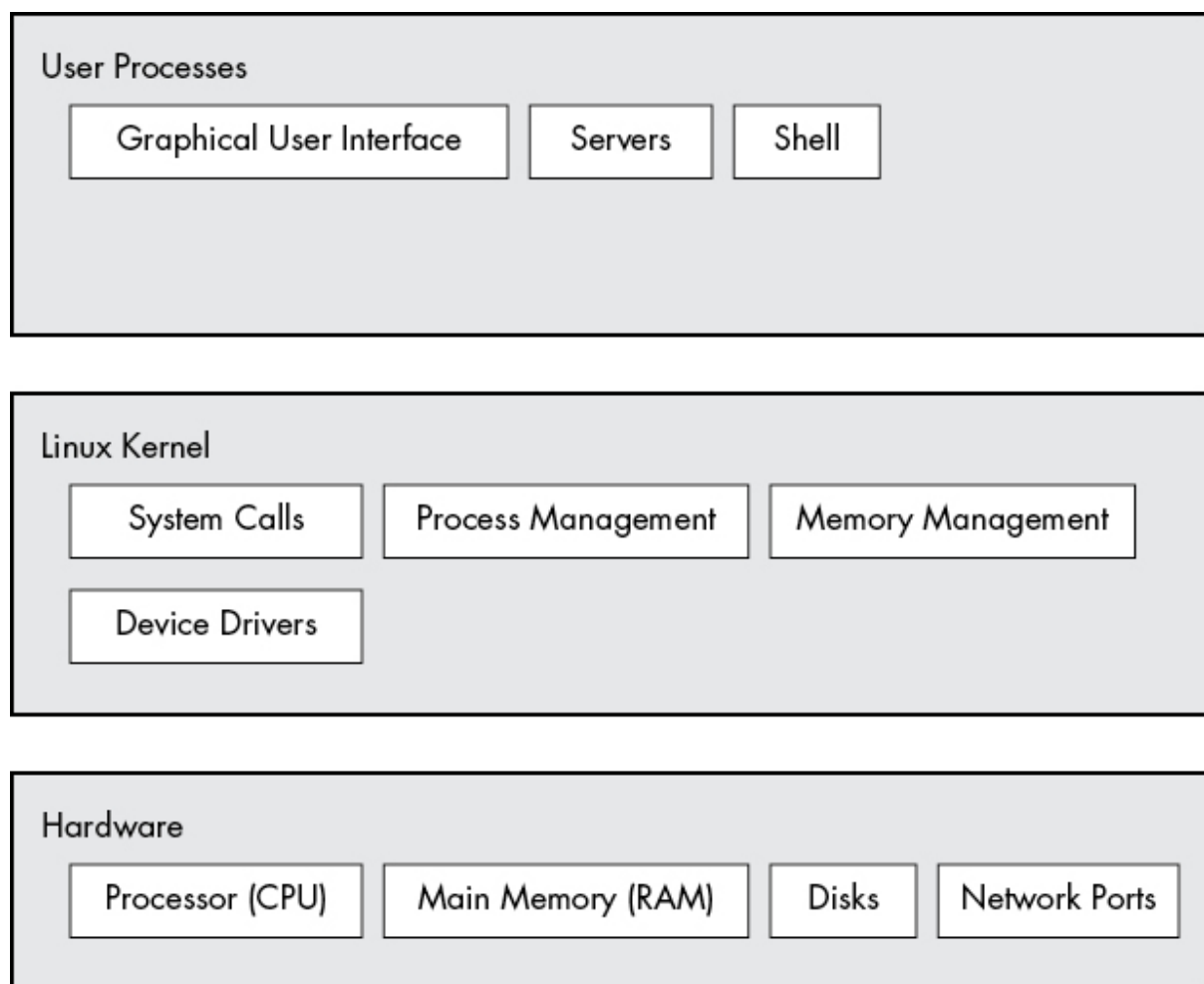


Levels and Layers of Abstraction in a Linux System

A Linux system has three main levels. The *hardware* is at the base. Hardware includes the memory as well as one or more central processing units (CPUs) to perform computation and to read from and write to memory. Devices such as disks and network interfaces are also part of the hardware.

The next level up is the *kernel*, which is the core of the operating system. The kernel is software residing in memory that tells the CPU where to look for its next task. Acting as a mediator, the kernel manages the hardware (especially main memory) and is the primary interface between the hardware and any running program.

Processes—the running programs that the kernel manages—collectively make up the system's upper level, called *user space*. (A more specific term for process is *user process*, regardless of whether a user directly interacts with the process. For example, all web servers run as user processes.)



There is a critical difference between how the kernel and the user processes run: the kernel runs in *kernel mode*, and the user processes run in *user mode*. Code running in kernel mode has unrestricted access to the processor and main memory. This is a powerful but dangerous privilege that allows the kernel to easily corrupt and crash the entire system. The memory area that only the kernel can access is called *kernel space*.

User mode, in comparison, restricts access to a (usually quite small) subset of memory and safe CPU operations. *User space* refers to the parts of main memory that the user processes can access. If a process makes a mistake and crashes, the consequences are limited and can be cleaned up by the kernel. This means that if your web browser crashes, it probably won't take down the scientific computation that has been running in the background for days.

Hardware: Understanding Main Memory

Of all of the hardware on a computer system, *main memory* is perhaps the most important. In its rawest form, main memory is just a big storage area for a bunch of 0s and 1s. Each slot for a 0 or 1 is called a *bit*. This is where the running kernel and processes reside—they're just big collections of bits. All input and output from peripheral devices flows through main memory, also as a bunch of bits. A CPU is just an operator on memory; it reads its instructions and data from the memory and writes data back out to the memory.

The Kernel

Nearly everything that the kernel does revolves around main memory. One of the kernel's tasks is to split memory into many subdivisions, and it must maintain certain state information about those subdivisions at all times. Each process gets its own share of memory, and the kernel must ensure that each process keeps to its share.

The kernel is in charge of managing tasks in four general system areas:

1. **Processes** The kernel is responsible for determining which processes are allowed to use the CPU.
2. **Memory** The kernel needs to keep track of all memory—what is currently allocated to a particular process, what might be shared between processes, and what is free.
3. **Device drivers** The kernel acts as an interface between hardware (such as a disk) and processes. It's usually the kernel's job to operate the hardware.
4. **System calls and support** Processes normally use system calls to communicate with the kernel.

We'll now briefly explore each of these areas.

Process Management

Process management describes the starting, pausing, resuming, scheduling, and terminating of processes. The concepts behind starting and terminating processes are fairly straightforward, but describing how a process uses the CPU in its normal course of operation is a bit more complex.

On any modern operating system, many processes run “simultaneously.” For example, you might have a web browser and a spreadsheet open on a desktop computer at the same time. However, things are not as they appear: the processes behind these applications typically do not run at *exactly* the same time.

Consider a system with a one-core CPU. Many processes may be *able* to use the CPU, but only one process can actually use the CPU at any given time. In practice, each process uses the CPU for a small fraction of a second, then pauses; then another process uses the CPU for another small fraction of a second; then another process takes a turn, and so on. The act of one process giving up control of the CPU to another process is called a *context switch*.

Each piece of time—called a *time slice*—gives a process enough time for significant computation (and indeed, a process often finishes its current task during a single slice). However, because the slices are so small, humans can't perceive them, and the system appears to be running multiple processes at the same time (a capability known as *multitasking*).

The kernel is responsible for context switching. To understand how this works, let's think about a situation in which a process is running in user mode but its time slice is up. Here's what happens:

1. The CPU (the actual hardware) interrupts the current process based on an internal timer, switches into kernel mode, and hands control back to the kernel.
2. The kernel records the current state of the CPU and memory, which will be essential to resuming the process that was just interrupted.
3. The kernel performs any tasks that might have come up during the preceding time slice (such as collecting data from input and output, or I/O, operations).
4. The kernel is now ready to let another process run. The kernel analyzes the list of processes that are ready to run and chooses one.
5. The kernel prepares the memory for this new process and then prepares the CPU.
6. The kernel tells the CPU how long the time slice for the new process will last.
7. The kernel switches the CPU into user mode and hands control of the CPU to the process.

The context switch answers the important question of *when* the kernel runs. The answer is that it runs *between* process time slices during a context switch.

In the case of a multi-CPU system, as most current machines are, things become slightly more complicated because the kernel doesn't need to relinquish control of its current CPU in order to allow a process to run on a different CPU, and more than one process may run at a time. However, to maximize the usage of all available CPUs, the kernel typically performs these steps anyway (and may use certain tricks to grab a little more CPU time for itself).

Memory Management

The kernel must manage memory during a context switch, which can be a complex job. The following conditions must hold:

- The kernel must have its own private area in memory that user processes can't access.
- Each user process needs its own section of memory.
- One user process may not access the private memory of another process.
- User processes can share memory.
- Some memory in user processes can be read-only.
- The system can use more memory than is physically present by using disk space as auxiliary.

Device Drivers and Management

The kernel's role with devices is relatively simple. A device is typically accessible only in kernel mode because improper access (such as a user process asking to turn off the power) could crash the machine. A notable difficulty is that different devices rarely have the same programming interface, even if the devices perform the same task (for example, two different network cards). Therefore, device drivers have traditionally been part of the kernel, and they strive to present a uniform interface to user processes in order to simplify the software developer's job.

System Calls and Support

There are several other kinds of kernel features available to user processes. For example, *system calls* (or *syscalls*) perform specific tasks that a user process alone cannot do well or at all. For example, the acts of opening, reading, and writing files all involve system calls.

Two system calls, `fork()` and `exec()`, are important to understanding how processes start:

1. `fork()` When a process calls `fork()`, the kernel creates a nearly identical copy of the process.
2. `exec()` When a process calls `exec(program)`, the kernel loads and starts program, replacing the current process.

Other than `init` *all* new user processes on a Linux system start as a result of `fork()`, and most of the time, you also run `exec()` to start a new program instead of running a copy of an existing process. A very simple example is any program that you run at the command line, such as the `ls` command to show the contents of a directory. When you enter `ls` into a terminal window, the shell that's running inside the terminal window calls `fork()` to create a copy of the shell, and then the new copy of the shell calls `exec(ls)` to run `ls`. shows the flow of processes and system calls for starting a program like `ls`.

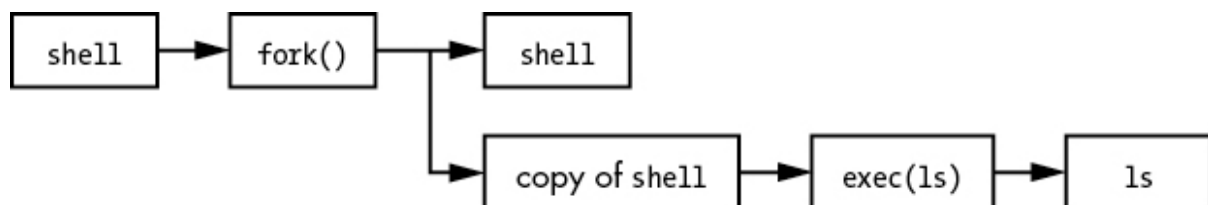


Figure: Starting a new process

User Space

As mentioned earlier, the main memory that the kernel allocates for user processes is called *user space*. Because a process is simply a state (or image) in memory, user space also refers to the memory for the entire collection of running processes.

Most of the real action on a Linux system happens in user space. Though all processes are essentially equal from the kernel's point of view, they perform different tasks for users.

Namespaces

Namespaces can be used for isolation.

Namespaces control what a process can see.

There are different namespaces like:

- Mount namespace
- Networking namespace
- PID namespace
- User namespace

When a process is kept in a namespace, it thinks it is the only process running on the server.

Even though you may have 500 programs running on the system, a program/process inside the namespace might only see 2 or 3 processes.

It might even think it is the root user, even though it has no power over the real system.

Cgroups (Control Groups)

While namespaces hide things, cgroups control how much a process can use.

It doesn't matter if a process thinks it's alone — if it starts using 100% of CPU, it will crash the whole system.

Cgroups prevent this by setting hard limits.

What do cgroups control?

- CPU → Limits processing power
- Memory → Limits RAM usage
- Disk I/O → Limits how fast it can read/write to the hard drive

Final Note

Containers use these features of namespaces and cgroups.